

# High-Quality Iterative Compiler for SIMD Logic-in-Memory Architecture with Tight Coupling of Synthesis and Scheduling

Xingyue Qian<sup>1</sup>, Chenyang Lv<sup>2</sup>, Zhezhi He<sup>2,\*</sup>, and Weikang Qian<sup>1,3,\*</sup>

<sup>1</sup>University of Michigan-SJTU Joint Institute, <sup>2</sup>School of Electronic Information and Electrical Engineering, and <sup>3</sup>MoE Key Lab of AI, Shanghai Jiao Tong University, Shanghai, China

Emails: {qianxingyue, lvchenyang, zhezhi.he, qianwk}@sjtu.edu.cn; \*corresponding authors

**Abstract**—Logic-in-memory (LiM) with single instruction multiple data (SIMD) setup enables memory to perform operations on the stored data in parallel to achieve high throughput and energy saving. To instruct a SIMD LiM hardware to compute a function, a compiler is needed that involves three steps: logic synthesis, scheduling, and instruction generation. Logic synthesis transforms the function into a netlist of supported operations. Scheduling determines the execution sequence of the operations in the netlist. Instruction generation maps the *scheduled netlist* into the instruction sequence given to the hardware. In this work, we propose an iterative LiM compiler with tight coupling of synthesis and scheduling to produce high-quality *scheduled netlists* that can generate instruction sequences with smaller energy-delay products (EDPs). It is based on improving the critical sub-netlist identified by our algorithm and performing domain-specific resubstitution. The experimental results show that our compiler can obtain better instruction sequences with EDPs reduced by 15.6% on average over the best existing method.

## I. INTRODUCTION

The data transfer between processor and memory in conventional von Neumann architecture is a performance bottleneck [1]. To reduce data transfer, *logic-in-memory (LiM)* is proposed, which enables memory to perform primitive logic operations such as majority (MAJ) and XOR on the stored data [2]–[5]. Compared to the other in-memory computing paradigms, LiM offers several advantages, including minimal hardware overhead and the support of general-purpose computing. *Single instruction multiple data (SIMD)* is a popular design style of LiM with high throughput [3]–[6], where all computation of a function, *e.g.*, addition, on an input pattern is performed in a single column of the memory array. In this way, by processing the cells in different columns of the same row *bitwise*, different columns can be used to compute the same function with different input patterns in parallel.

To instruct a SIMD LiM hardware to compute a function on the stored data, a *compiler* is needed to obtain the *instruction sequence (IS)* given to the hardware [5], [7]–[12]. An instruction either performs a supported operation on some rows within a memory array or does some data management such as copying one row to another, and one instruction is performed per clock cycle. Different ISs have different energy-delay products (EDPs). A *typical objective of a LiM compiler is to find an IS with a low EDP*. Note that the compiler is run *offline* to obtain the IS.

The basic flow of a LiM compiler is shown in Fig. 1(a). It has three steps: *logic synthesis*, *scheduling*, and *instruction*

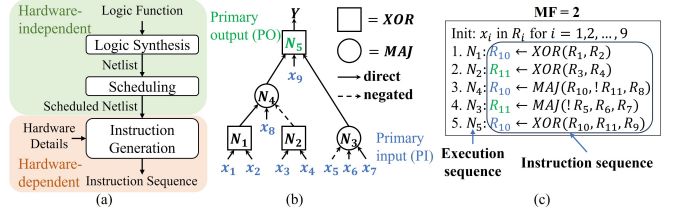


Fig. 1. SIMD LiM compiler: (a) the basic flow; (b) a netlist example. Note that some operations in the figure only have two fan-ins, because we omit the constant 0 for simplicity; (c) an example of execution sequence and instruction sequence.

*generation*. Logic synthesis converts the target function into a netlist of the supported operations, *e.g.*, XOR-majority graph (XMG) as shown in Fig. 1(b). Since it takes time and energy to perform each operation in the netlist in serial in the SIMD setup, a target of logic synthesis is to minimize the *size* of the netlist, *i.e.*, the number of operations.

Then, the second step, scheduling, determines the *execution sequence* of the operations. For example, the execution sequence shown in Fig. 1(c) is  $N_1, N_2, N_4, N_3$ , and  $N_5$ . Thus, the output of the scheduling step can also be viewed as a netlist with an execution order of all nodes in the netlist. For simplicity, we call it a *scheduled netlist*. At each clock cycle of performing an operation, there are some operations whose results must be stored in memory rows for future use, which are called *live operations*. The maximum number of live operations at each clock cycle over the entire process is called *memory footprint (MF)*. MF is an important metric of a scheduled netlist since it is the *minimum number of memory rows needed to perform the operations* according to the execution sequence [10]. A common target of the scheduling step is to minimize MF [5], [7]–[12], since a scheduled netlist with less MF is more likely to lead to an IS with less EDP after the instruction generation step [5], [10].

The last step, instruction generation, generates an IS based on the scheduled netlist according to current hardware details such as the number of available rows in an array. In the ideal case where we can compute the function in a single array, the execution sequence is mapped to IS by generating one instruction for each operation as shown in Fig. 1(c). When multiple arrays are needed, the costly cross-array data copying instructions are inserted when needed.

Most existing LiM compilers are based on the above basic flow [7]–[12]. In the logic synthesis step, the function is first converted into a supported netlist by logic synthesis tools such as ABC [13], and then a sequence of logic optimization

commands is applied to the netlist to reduce its size. In the scheduling step, the optimized netlist is scheduled using a heuristic [7]–[9] or a divide-and-conquer [10] approach to minimize MF. In the final instruction generation step, naive [5] or greedy [10] approaches are used to obtain an IS by taking the hardware details into account. These basic compilers [7]–[12] are relatively fast, but *they can only produce a single IS with sub-optimal performance*. Besides, *they make no attempt to improve the performance of the IS*. Recently, reinforcement learning (RL) is used to obtain a sequence of optimization commands that is applied to a baseline netlist, in order to improve the EDP of the IS it generates [5]. However, due to the long runtime of the RL algorithm, the sequence only contains 10 commands, and the scheduler used in the process is simple. Hence, the obtained IS is still unsatisfactory.

To address the above issues, we propose an iterative LiM compiler with a tight coupling of logic synthesis and scheduling that can obtain high-quality scheduled netlists. Since the scheduled netlists are independent of hardware details, we do not need to recompile when the hardware status changes. Given current hardware status, we use the greedy instruction generation approach in [10] to obtain the corresponding ISs for the scheduled netlists. For simplicity, in what follows, we sometimes call the scheduled netlist a *design* of the given function. Our contributions are:

- For the first time, we reveal that a netlist with a smaller size does not necessarily have less MF after scheduling. Based on this fact and the fact that a design with a smaller size and MF tends to lead to an IS with a smaller EDP after the instruction generation step, we propose to find Pareto-optimal designs in terms of size and MF.
- We propose a flow that iteratively picks a Pareto-optimal design obtained so far and attempts to improve its performance by re-synthesizing and re-scheduling it.
- To speed up the iterative flow, we propose a technique exploiting the scheduling result to extract a critical sub-netlist for re-synthesis instead of the whole netlist.
- To further improve the performance of the compiler, we propose a powerful logic optimization method based on traditional resubstitution [14], which directly targets at reducing MF by exploiting the scheduling result.
- We make the code of our compiler open-source at <https://github.com/SJTU-ECTL/IMCCompiler>.

The experimental results show that our compiler can obtain better Pareto-optimal designs on size and MF, and the EDPs of the resulting ISs are reduced by 15.6% on average compared to the best state-of-the-art method.

## II. PRELIMINARIES

### A. Netlist Representation and Basic Scheduling Rules

In this work, we use the architecture proposed in [5] as our target architecture. It supports inversion and 3-input XOR and MAJ operations, where the inversion is achieved by directly reading the negated output of a memory cell. Hence, the corresponding netlist is an XMG as shown in Fig. 1(b). Each node in the netlist corresponds to either an XOR or a MAJ operation, and we use the terms operation

and node interchangeably. A directed edge from node  $i$  to node  $j$  indicates that the output of node  $i$  is an input of node  $j$ . If the edge is dashed, the output of node  $i$  is negated. We call node  $i$  a *fan-in* of node  $j$  and node  $j$  a *fan-out* of node  $i$ . The *primary inputs (PIs)* of the function are initially stored in memory and cannot be deleted during the process, since they may still be needed in the future. For the *primary outputs (POs)* of the function, their results must be in memory when the computation ends, and thus also cannot be deleted once computed. *The results of other operations can be deleted from the memory once they are no longer needed to save memory space*. Here, by deleting an operation result from memory, we mean that the row containing the result can be overwritten and hence, becomes available for future operations. We call an operation whose result must be stored in a memory row at clock cycle  $i$  for future use a *live operation* at clock cycle  $i$ . Hence, an operation becomes *live* once it is scheduled. If it is a PO, it remains live until the computation ends. Otherwise, it remains live until all its fan-outs have been scheduled.

An operation can be executed by passing an instruction to the hardware when all its fan-ins are stored in memory. In the example in Fig. 1(c), the execution sequence is  $N_1, N_2, N_4, N_3, N_5$ , and the corresponding instruction to execute each operation is listed to the right of “:”. For example, the instruction at clock cycle three, which performs operation  $N_4$ , is  $R_{10} \leftarrow MAJ(R_{10}, !R_{11}, R_8)$ , meaning that we perform an MAJ operation on row 10, negated row 11, and row 8, which contain the results of operation  $N_1$ , operation  $N_2$ , and PI  $x_8$ , respectively, and store the result in row 10.

However, an operation can only be applied to the rows in the same memory array, but the number of rows in an array is limited [15]. When the rows containing the fan-ins of an operation are at different arrays, we need to execute extra costly cross-array data copying instructions before executing the operation. This may diminish the advantage of LiM [5], [8], [16]. Hence, a common target of a scheduler is to minimize the MF of the scheduled netlist [5], [7]–[11], since it is the minimum number of rows required to perform the operations according to the execution sequence. By minimizing MF, we can compute functions with larger netlists within a single array to reduce EDP. Also, when multiple arrays are required, the designs with less MF tend to generate ISs with fewer copy instructions after instruction generation, thus having less EDP [10].

Given a scheduled netlist, if the MF is no more than the number of available rows in an array, we can compute the function using a single array. Note that a row becomes available once the operation result stored in it is no longer needed. In other words, the operation is no longer live. In this case, for the operation scheduled at each clock cycle, the instruction generation process chooses to store its result at the row available at the clock cycle with the smallest index [10]. If the MF is more than the number of available rows in an array, we must use multiple arrays to compute the function. The instruction generation process is more complicated as we have to insert cross-array data copying instructions when needed. In this work, we use the greedy process proposed in [10] for this case since it outperforms the naive method in [5].

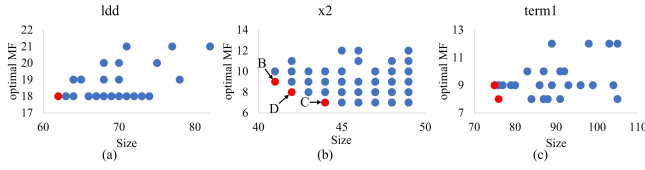


Fig. 2. Design space of (a) ldd, (b) x2, and (c) term1.

### B. Resubstitution

Resubstitution is a powerful method in logic synthesis for reducing the size of a netlist [14]. Its main idea is to check whether the function of a node, called *root*, in the netlist can be expressed using other existing nodes, called *divisors*. If so, we substitute the root with the expression we find, and all nodes that are used exclusively by the root, *i.e.*, its *maximum fanout free cone (MFFC)* [17], can be deleted, thus reducing the size of the netlist. A state-of-the-art resubstitution method is the simulation-guided resubstitution [14]. It traverses the netlist, treating each node as the root, and collects a subset of the nodes in the netlists as its potential divisors. It checks for the existence of an expression based on logic simulation followed by validation via a satisfiability solver.

## III. MOTIVATION

An ultimate goal for a LiM compiler is producing an IS with a small EDP for the given hardware details. As we mentioned in Sections I and II-A, the basic flow of a LiM compiler [7]–[12] minimizes the size of the netlist in the logic synthesis step and then the MF of the scheduled netlist in the scheduling step, since a design with a smaller size and MF tends to lead to an IS with a smaller EDP after the instruction generation step.

However, a further question is whether a netlist with a smaller size always has less MF after scheduling. If so, we can apply the basic flow of minimizing size and MF in serial to obtain the best design with the least size and MF. To answer this question, we randomly generate some netlists for three small benchmarks in LGSynth89 [18], *ldd*, *x2*, and *term1*, and use the optimal scheduler proposed in [10] to obtain the optimal MF, eliminating the impact of different scheduling algorithms. We plot the sizes and the MFs of various designs for each benchmark in Fig. 2, where each point corresponds to a design. For benchmark *ldd*, there exists one best design marked in red. However, for benchmarks *x2* and *term1*, there exist multiple designs marked in red with size-MF tradeoff at the Pareto frontier. This tradeoff is reasonable since for a given function, a netlist with a smaller size tends to be more densely connected. That is, a node in the netlist has more fan-outs. Hence, the node should stay live for a longer time until all its fan-outs are scheduled, leading to a larger MF. *In conclusion, a design with a smaller size does not necessarily have less MF.* In other words, there typically exists a trade-off between size and MF. Given that a design with a smaller size and MF tends to lead to an IS with a smaller EDP after the instruction generation step, we reformulate the problem of identifying an IS with a small EDP for a given function into the problem of *finding the Pareto-optimal designs on size and MF for the function*. Once the Pareto-optimal designs are obtained, we can use the following process to quickly generate

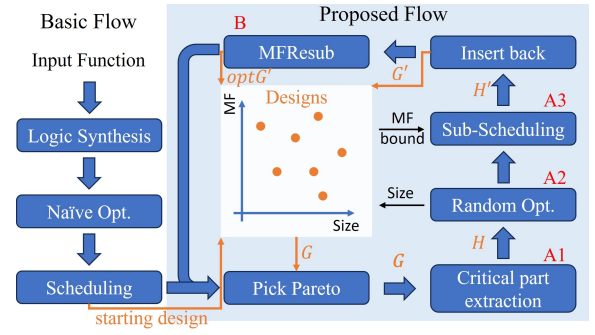


Fig. 3. The flow of our iterative LiM compiler.

the IS given the hardware details without the need to recompile when the hardware details change. If the least MF among all Pareto-optimal designs is no larger than the number of available rows in the target array, we can use a single array for high throughput. Thus, we pick the Pareto-optimal design with the least size while having MF no larger than the number of available rows. The corresponding IS can be generated using the instruction generation process for a single array mentioned in Section II-A. Otherwise, we must use multiple arrays, and we use the greedy instruction generation method from [10] to generate the corresponding IS for each Pareto-optimal design and pick the IS with the least estimated EDP.

We use the following example to show the advantage of our new formulation by comparing its solution over that of the basic flow of a LiM compiler.

**Example 1** *In the example in Fig. 2(b), the best design found by the basic flow, *i.e.*, first minimizing size and then minimizing MF, is design B. It has 41 operations and an MF of 9. If we use arrays with 8 available rows for the computation, it requires two arrays and at least one copy instruction. In contrast, our compiler can find the Pareto-optimal designs marked in red. When using arrays with 8 available rows, we first check for the possibility of using a single array to achieve high throughput and low EDP. We pick design D since among all Pareto-optimal designs with MF no larger than 8, *i.e.*, design C and design D, design D has a smaller size. Although design D has one more operation than design B, its MF is only 8, meaning that it only requires one array and no copy instructions. Although a copy instruction takes one additional clock cycle like an operation instruction, it consumes much more energy than an operation instruction [5], so design D has less EDP than design B found by the basic compiler.*

## IV. METHOD

This section presents our LiM compiler that obtains the Pareto-optimal designs on size and MF. To facilitate the illustration, we *reorder the indices of the operations in the scheduled netlist according to the execution sequence*, where operation with index  $i$  is the  $i$ -th operation to be executed. We use  $Size(G)$  and  $MF(G)$  to denote the size and MF of a scheduled netlist (or design)  $G$ , respectively.

Fig. 3 shows the flow of our LiM compiler. First, the compiler executes a basic compiling flow to obtain a starting design. Specifically, the input function is synthesized to an XMG netlist using Mockturtle [19] and then optimized using

the optimization commands in the tool to reduce the netlist size. Then, the optimized netlist is scheduled with the state-of-the-art scheduler [10], and the resulting design is added into the design space. Now, instead of directly generating the IS for this design as in most prior works [7]–[11], we try to *find designs of better quality in terms of Pareto-optimality*. The process is shown in the light blue box in Fig. 3, and is run for  $R$  rounds. In each round, we randomly pick a baseline design  $G$  from the Pareto-optimal designs of the current explored design space and try to improve it by modifying its netlist and re-scheduling it. The improved design is added to the design space to update the Pareto-optimal frontier. We pick from Pareto-optimal designs since they are the ones with the highest quality so far and thus, are promising candidates for optimization to advance the Pareto frontier. After the iterative improvement process ends, we obtain the Pareto-optimal designs (*i.e.*, scheduled netlists). Then, we use the process introduced in Section III to quickly generate the IS for the given hardware details. Note that if the hardware details change, we only need to regenerate the IS based on the same set of Pareto-optimal designs, instead of compiling from scratch.

To accelerate the entire compiling flow, in each round, we first exploit the scheduling information to extract a *critical sub-netlist*  $H$  from the baseline design  $G$  that is crucial for quality improvement, and attempt to re-synthesize and re-schedule it instead of the whole netlist. The optimized critical sub-netlist  $H'$  will be inserted back into  $G$  to form a new design  $G'$ . The details of the critical sub-netlist optimization will be introduced in Section IV-A.

To further improve the quality of the design, we propose *MFResub* in Section IV-B, a resubstitution method oriented at MF reduction by taking scheduling information into account. We apply *MFResub* to the newly generated design  $G'$  and produce a design  $optG'$ , which is further added into the design space to update the Pareto frontier. Note that *MFResub* is applied to the whole netlist instead of the critical sub-netlist, since it is relatively fast and we can find more optimization opportunities on the whole netlist.

#### A. Critical Sub-netlist Optimization

In order to efficiently improve an existing design by re-synthesis and re-scheduling, we propose to work on a critical sub-netlist instead of the whole netlist. In the following, we will first explain how to identify and extract the critical sub-netlist in Section IV-A1. Then, we will illustrate how to optimize and schedule it in Sections IV-A2 and IV-A3, respectively. After we get the optimized and re-scheduled sub-netlist, we will replace the old critical sub-netlist in the original design with it to get a new design for the input function.

1) *Critical Sub-netlist Extraction*: To identify the critical sub-netlist, we first define *memory usage* at clock cycle  $i$  as the number of live operations at the end of clock cycle  $i$  after operation  $i$  is scheduled. By definition, *MF* equals the *maximum memory usage over the entire process*. In Fig. 4(a), we plot the memory usage versus clock cycle of a design of benchmark *log2*. We can see that during the entire period, the memory usage starts at one at clock cycle one, reaches the

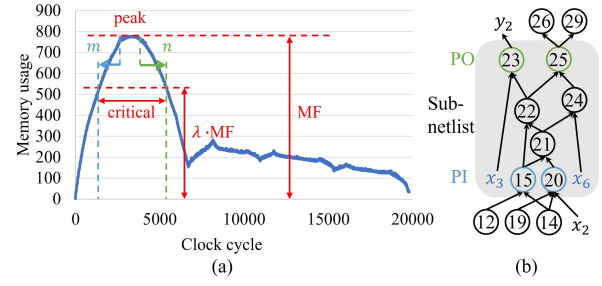


Fig. 4. Critical sub-netlist extraction: (a) memory usage and critical sub-netlist of *log2* benchmark; (b) an example critical sub-netlist.

maximum value, which equals the MF of the design, and ends at a value equal to the PO number.

We define *peak period* as the period from clock cycle  $m$  to clock cycle  $n$ , where  $m$  and  $n$  are the first and last clock cycles at which the memory usage reaches MF, respectively, as shown in Fig. 4(a). Since MF equals the maximum memory usage during the entire process, which is reached during the peak period, *if we can decrease the maximum memory usage during the peak period, we decrease the MF of the whole netlist*. Since working with the operations scheduled during the peak period only is too restrictive, we expand the peak period to include some cycles before and after it. Specifically, as illustrated in Fig. 4(a), we decrease the value of  $m$  and increase the value of  $n$  until the memory usage at clock cycles  $m$  and  $n$  drops below  $\lambda \cdot MF$ , where  $0 < \lambda < 1$  is called the *critical parameter*.

The critical sub-netlist  $H$  are constructed with the operations scheduled between the above updated clock cycles  $m$  and  $n$ , *i.e.*, those with indices ranging from  $m$  to  $n$ . Besides, some nodes in the entire netlist  $G$  should be added to serve as its PIs, and some nodes of the sub-netlist should be identified as its POs. We illustrate the construction process by an example.

**Example 2** In Fig 4(b), we assume that  $m = 21$  and  $n = 25$ . We construct the sub-netlist  $H$  in the gray box whose operations are those indexed from 21 to 25 in the design  $G$ . The PIs of the entire netlist  $G$  and the operations scheduled before clock cycle 21 that are the fan-ins of the operations in  $H$ , *i.e.*,  $x_3$ ,  $x_6$ , operation 15, and operation 20, are included in  $H$  as its PIs. The POs of  $H$  are the operations of  $H$  that have fan-outs outside the sub-netlist, *i.e.*, operation 25, and operations that are POs of  $G$ , *i.e.*, operation 23.

We can modify the sub-netlist  $H$  constructed in the above way by optimization commands without changing the original function as long as the functions of the POs of  $H$  are unchanged. In this way, *we can work on  $H$  whose performance is crucial but with a much smaller size*.

2) *Sub-netlist Optimization*: We apply  $K$  random optimization commands in Mockturtle [19] including resubstitution [17] and rewriting [20], [21] to the critical sub-netlist  $H$ , hoping to obtain a functionally equivalent netlist  $H'$  with better quality. To further reduce the size, we additionally apply the powerful command sequence *resyn2* in ABC [13] to  $H'$ , using the corresponding commands in Mockturtle [19].

3) *Sub-netlist Scheduling*: We apply the scheduler from [10] to obtain the execution sequence of  $H'$ . During the



scheduling, the PIs of  $H'$  that are the operations of  $G$ , e.g., operations 15 and 20 in Fig. 4(b), are regarded as *temporary inputs* of the sub-netlist defined in [10], and they can be deleted from memory when they are no longer needed. After scheduling, the indices of the operations in  $H'$  are also reordered according to the execution sequence.

To accelerate the process, we adopt an early-termination rule in the scheduling process. Specifically, after we obtain the optimized sub-netlist  $H'$ , we can calculate the size of the netlist  $G'$ , which will be obtained later by substituting the critical sub-netlist  $H$  of the original netlist  $G$  with  $H'$ . It is

$$\text{Size}(G') = \text{Size}(G) - \text{Size}(H) + \text{Size}(H').$$

Since we want to find the Pareto-optimal designs, we can obtain an upper bound of MF so that  $G'$  can be a new Pareto-optimal design. Specifically, among all Pareto-optimal designs we have found so far, we identify the one, denoted as  $D$ , with size no more than  $\text{Size}(G')$  and closest to  $\text{Size}(G')$ . Then, we require the MF of  $G'$  to be less than  $\text{MF}(D)$ .

**Example 3** For example, suppose that the designs we have explored so far are those shown in Fig. 2(b) and that the size of  $G'$  is 43. Among all Pareto-optimal designs we have found so far, i.e., the red points, the one with size no more than  $\text{Size}(G')$  and closest to  $\text{Size}(G')$  is the design  $D$  shown in Fig. 2(b). Since its MF is 8, we require that the MF of  $G'$  be less than 8 if it would become a Pareto-optimal design.

Since we will apply our MF-oriented resubstitution to  $G'$  later to further improve its quality, we relax this upper bound in scheduling stage by multiplying it with a *bound relaxation parameter*  $\beta > 1$ . The above bound can be used in the scheduler in [10] to achieve the early termination. If the bound is not achievable, we terminate the scheduler immediately and begin another round of the compiler flow without executing the remaining steps in this round.

### B. MF-Oriented Resubstitution

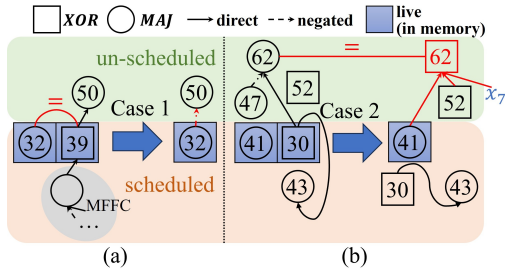


Fig. 5. The two cases of MF-oriented resubstitution: (a) case 1 where there exist two functionally equivalent live operations, 32 and 39; (b) case 2 where we can resubstitute the only un-scheduled fan-out of operation 30 to make operation 30 no longer live.

To further improve the quality of a design, we propose *MFResub*, a resubstitution method oriented at MF reduction by taking the scheduling information into account. The pseudocode is shown in Algorithm 1, and we will explain it using the two examples in Fig. 5. The inputs are a scheduled netlist  $G$  and a limit on the number of trials  $N_{\text{trial}}$ . The output is an optimized scheduled netlist  $\text{opt}G$ . Line 1 obtains  $p$ , the first clock cycle at which the memory usage reaches the maximum value, i.e., the MF of the scheduled netlist. Line 2 obtains the

### Algorithm 1: The flow of MF-oriented resubstitution.

---

**Input:**  $G$ : a scheduled netlist;  $N_{\text{trial}}$ : a limit on the number of trials.

**Output:**  $\text{opt}G$ : an optimized scheduled netlist.

```

1  $p \leftarrow \text{FirstPeak}(G)$ ;
2  $\mathcal{P} \leftarrow \text{OpInMem}(p)$ ;
3 foreach  $j \in \mathcal{P}$  do
4   foreach  $i \in \mathcal{P}$  with  $i < j$  do
5      $s \leftarrow \text{CheckSub}(G, j, i)$ ;
6     if  $s$  is not NULL then return  $\text{SubOp}(G, j, s)$ ;
7 foreach  $j \in \mathcal{P}$  do
8   if  $\text{isPO}(j)$  or  $\text{NoSingFO}(j, p)$  then continue;
9    $f \leftarrow \text{GetSingFO}(j, p)$ ;
10   $\text{Divisors} \leftarrow (\mathcal{P} \setminus \{j\}) \cup \{k \mid p < k < f\}$ ;
11   $\text{Divisors} \leftarrow \text{PruneDiffPI}(\text{Divisors}, f) \cup \text{GetPI}(f)$ ;
12  if  $\text{NumEnum}(|\text{Divisors}|) \leq N_{\text{trial}}$  then
13     $\text{DivTrip} \leftarrow \text{Enum}(\text{Divisors})$ ;
14  else  $\text{DivTrip} \leftarrow \text{Rand}(\text{Divisors}, N_{\text{trial}})$ ;
15  foreach  $(u, v, w) \in \text{DivTrip}$  do
16     $s \leftarrow \text{CheckSubOp}(G, f, (u, v, w))$ ;
17    if  $s$  is not NULL then return  $\text{SubOp}(G, f, s)$ ;
18 return  $G$ ;
```

---

live operations at the end of clock cycle  $p$  after operation  $p$  is scheduled and stores them in a set  $\mathcal{P}$ . By definition, the size of  $\mathcal{P}$  equals the MF of the scheduled netlist. If we can make an operation in  $\mathcal{P}$  no longer live at clock cycle  $p$  by resubstitution, we may decrease the MF of the whole netlist. In the examples in Fig. 5, we assume that  $p = 45$ , and the live operations in the set  $\mathcal{P}$  are marked in the blue boxes. We attempt to find the following two cases of resubstitution.

1) *Case 1*: Lines 3–6 correspond to case one as shown in Fig. 5(a). In case one, two live operations are functionally equivalent, so we can directly remove one of them from the netlist. Line 5 checks whether an operation  $j$  in  $\mathcal{P}$  is functionally equivalent to another (possibly negated) operation  $i$  in  $\mathcal{P}$  with smaller index with a function *CheckSub*, which uses the checking process of the simulation-guided resubstitution [14] described in Section II-B. If so, *CheckSub* returns  $s$  as  $i$  or its negation according to the equivalence result. Otherwise, it returns  $s$  as NULL. If  $s$  is not NULL, function *SubOp* at Line 6 substitutes the operation  $j$  with  $s$  and removes all operations in the MFFC of  $j$ . As  $j$  is removed from the netlist, it is also removed from  $\mathcal{P}$ , which may lead to a reduction in MF.

**Example 4** In the example in Fig. 5(a), among the live operations, operation 39 is functionally equivalent to the negation of operation 32. Hence, we substitute operation 39 with the negation of operation 32 when it serves as a fan-in of node 50. After the substitution, we remove node 39 from the netlist together with its MFFC, leading to a reduction in netlist size and the current memory usage, which may lead to a reduction in MF.

2) *Case 2*: Lines 7–17 correspond to case two as shown in Fig. 5(b). In case two, we cannot remove an operation from the netlist, but we can resubstitute its fan-out to make its result not needed by any un-scheduled operations. Thus, it is no longer live and can be deleted from  $\mathcal{P}$ . We iterate over

all operations  $j \in \mathcal{P}$  and focus on the operation that is not a PO and has only one un-scheduled fan-out at clock cycle  $p$ . Line 8 prunes the unqualified operations. For a qualified operation, Line 9 obtains its single unscheduled fan-out  $f$ . If we can substitute  $f$  with a new operation that does not take  $j$  as its fan-in, the result of  $j$  is no longer needed. Thus,  $j$  is no longer live at clock cycle  $p$  and can be deleted from  $\mathcal{P}$ . The alternative fan-ins of this new operation are picked from a set *Divisors* constructed in Lines 10–11. Since we do not want to change the execution sequence of the netlist, Line 10 initially constructs the set to include all the other live operations at clock cycle  $p$ , i.e.,  $\mathcal{P} \setminus \{j\}$ , and the operations to be scheduled from clock cycle  $(p+1)$  to clock cycle  $(f-1)$ . Then, Line 11 refines the set based on PI dependency. We say that an operation depends on a PI if there is a directed path from the PI to the operation. The function of an operation is entirely determined by the PIs on which it depends. If an operation  $d \in \text{Divisors}$  depends on a PI  $u$ , but the operation  $f$  does not depend on  $u$ , it is unlikely that we can find a functionally equivalent new operation with  $d$  as its fan-in to substitute  $f$ . Hence, Line 11 prunes these divisors by the function *PruneDiffPI*. Besides, it also puts the PIs on which  $f$  depends and the constant signal into *Divisors*.

Next, since our netlist is XMG where each operation has three inputs, we need to check whether each triple  $(u, v, w)$ , where  $u, v, w \in \text{Divisors}$ , can be used as the fan-ins of the new operation to substitute  $f$ . We put all triples we will try into a set *DivTrip* and bound its size with a parameter  $N_{\text{trial}}$ . If the number of different triples in *Divisors* is no larger than  $N_{\text{trial}}$ , we enumerate them in Line 13. Otherwise, we randomly form  $N_{\text{trial}}$  triples in Line 14. For each triple  $(u, v, w) \in \text{DivTrip}$ , we use the function *CheckSubOp* to check whether a new operation  $s$  of the supported type, i.e., MAJ or XOR, can be constructed using (possibly negated)  $u, v, w$  as its fan-ins that is functionally equivalent to the operation  $f$  in Line 16. The function *CheckSubOp* also uses the checking process in [14] to check for the equivalence. If such an operation  $s$  can be constructed, i.e.,  $s$  returned by *CheckSubOp* is not *NULL*, we substitute the operation  $f$  with  $s$  in Line 17. Thus, we can remove  $j$  from  $\mathcal{P}$ .

**Example 5** In the example in Fig. 5(b), operation 62 is functionally equivalent to a newly constructed XOR operation taking live operation 41, operation 52 to be scheduled before operation 62, and PI  $x_7$  as fan-ins. Thus, we substitute operation 62 with the newly constructed operation (also indexed 62). Since the original operation 62 is the only un-scheduled fan-out of operation 30, operation 30 is no longer needed for the future scheduling after the substitution. Hence, it is no longer live, leading to a reduction in memory usage. Note that the MFFC of the original operation 62 can be removed from the netlist, leading to a reduction in netlist size.

Note that during the process, the relative order of the indices of the operations left in the scheduled netlist is kept, so we can simply re-assign the indices to skip those of the removed operations without re-scheduling the netlist. In our implementation, if a resubstitution is found, the algorithm immediately returns the modified scheduled netlist (see Lines 6

Table I. Benchmarks used in the experiment.

| Name      | #Nodes | #PI | #PO | Name       | #Nodes | #PI | #PO |
|-----------|--------|-----|-----|------------|--------|-----|-----|
| router    | 257    | 60  | 3   | sin        | 5416   | 24  | 25  |
| int2float | 260    | 11  | 7   | sqrt       | 24618  | 128 | 64  |
| cavlc     | 693    | 10  | 11  | multiplier | 27062  | 128 | 128 |
| priority  | 978    | 128 | 8   | log2       | 32060  | 32  | 32  |
| max       | 2865   | 512 | 130 | div        | 57247  | 128 | 128 |

and 17), and the entire algorithm is applied on the the modified netlist again. This process repeats until no resubstitution is found. In this case, the algorithm returns the input design in Line 18.

## V. EXPERIMENTAL RESULTS

This section shows the experimental results. We implement our LiM compiler in C++ and perform experiments on a computer with a 24-core 2.4GHz Intel 4214R processor and 64GB RAM. Our target hardware is the one proposed in [5], and we use the method in [5] to estimate the EDPs of the ISs. Note that *our compiler can be easily adapted to support other LiM hardware*s as long as they satisfy two requirements: 1) a logic operation can be performed on any rows of the memory array and 2) a logic operation does not change the values of the inputs to the operation. Many prevalent LiM hardware such as MAGIC [2] satisfy the above requirements. In order to support another LiM hardware, we only need to modify the supported operations in *MFResub* and use a proper logic synthesis tool that supports the netlist type, e.g., NOR2 graph of the MAGIC architecture [2], without changing our algorithm.

The benchmarks used in our experiments are from the EPFL benchmark suite [22], which is also used in prior works [5], [8]–[10]. The information of the benchmarks is listed in Table I. #Nodes is the number of nodes in the AND-inverter graph (AIG) provided in the benchmark suite, and constant POs are not counted in #POs. The inputs to a LiM compiler include the AIG file of a benchmark and the array size, i.e., the number of rows in an array, and the output is the IS given to the LiM hardware to compute the benchmark. As mentioned in Section I, a LiM compiler is run *offline* to obtain the IS.

We set the number of rounds for iterative improvement  $R = 20$ , the critical parameter  $\lambda = 0.6$ , the bound relaxation parameter  $\beta = 1.1$ , the number of optimization commands  $K = 10$ , and the limit on the number of trials  $N_{\text{trial}} = 500000$ . We compare our compiler to the following compilers.

- GPPIC [5] and GPPIC+: GPPIC+ is an improved version of GPPIC. Since the scheduler used in our work is better than that used in GPPIC, we re-schedule the netlist obtained by GPPIC with the scheduler we use, i.e., [10], and call the combined approach GPPIC+.
- LiM compilers that are based on basic compiling flows (BCs), i.e., [7]–[10]: These compilers target at different hardware architectures that support different types of netlists. In our comparison, we modify them to support XMG without changing their flows and algorithms.
- A counterpart of our compiler (CTP): To show the strength of our flow featuring coupled synthesis and scheduling, we develop a counterpart of our compiler, i.e., CTP. CTP also iteratively optimizes the Pareto-optimal designs, but it performs the netlist optimization step in

Table II. The sizes and MFs of the designs obtained by various compilers and the EDPs ( $10^{-15}$ J.s) of the resulting ISs.

| Benchmark  | ArraySize | GPPIC [5] |      |        | GPPIC+ |     |               | Best BC ([7]–[10]) |     |              | CTP   |     |        | Ours  |     |               | Runtime (h) |
|------------|-----------|-----------|------|--------|--------|-----|---------------|--------------------|-----|--------------|-------|-----|--------|-------|-----|---------------|-------------|
|            |           | Size      | MF   | EDP    | Size   | MF  | EDP           | Size               | MF  | EDP          | Size  | MF  | EDP    | Size  | MF  | EDP           |             |
| router     | 64        | 197       | 41   | 0.0031 | 197    | 10  | 0.0029        | 201                | 16  | 0.0026       | 204   | 12  | 0.0026 | 196   | 8   | <b>0.0024</b> | 0.22        |
| int2float  | 16        | 209       | 45   | 0.0016 | 209    | 19  | 0.0013        | 199                | 18  | 0.0011       | 205   | 16  | 0.0013 | 208   | 14  | <b>0.0011</b> | 0.08        |
| cavlc      | 64        | 592       | 151  | 0.0178 | 592    | 70  | 0.0116        | 600                | 60  | 0.0070       | 662   | 42  | 0.0070 | 596   | 49  | <b>0.0052</b> | 0.18        |
| priority   | 128       | 535       | 112  | 0.0176 | 535    | 28  | <b>0.0171</b> | 551                | 17  | 0.0184       | 541   | 23  | 0.0179 | 545   | 18  | 0.0171        | 0.32        |
| max        | 256       | 1935      | 336  | 0.93   | 1935   | 263 | 0.77          | 1824               | 264 | 0.72         | 2003  | 264 | 0.78   | 1820  | 263 | <b>0.71</b>   | 0.44        |
| sin        | 256       | 3619      | 344  | 1.01   | 3619   | 262 | 0.87          | 3482               | 264 | 0.81         | 3540  | 252 | 0.80   | 3524  | 222 | <b>0.63</b>   | 1.32        |
| sqrt       | 256       | 9699      | 194  | 11.16  | 9699   | 191 | 9.52          | 9240               | 275 | 15.34        | 9997  | 191 | 9.72   | 9166  | 188 | <b>8.12</b>   | 2.16        |
| multiplier | 256       | 14251     | 1414 | 65.78  | 14251  | 261 | 33.68         | 14176              | 247 | <b>25.88</b> | 14303 | 262 | 30.63  | 14303 | 253 | 26.31         | 4.71        |
| log2       | 256       | 21276     | 1283 | 81.71  | 21276  | 763 | 46.64         | 19760              | 789 | 43.35        | 19821 | 771 | 41.81  | 19899 | 722 | <b>41.16</b>  | 3.92        |
| div        | 256       | 28379     | 491  | 128.93 | 28379  | 272 | 92.43         | 12536              | 268 | 27.13        | 15272 | 377 | 34.27  | 12533 | 248 | <b>21.01</b>  | 4.79        |
| GEOMEAN    |           | 2445      | 240  | 0.64   | 2445   | 110 | 0.48          | 2206               | 111 | 0.40         | 2324  | 106 | 0.40   | 2210  | 93  | 0.34          | 0.81        |

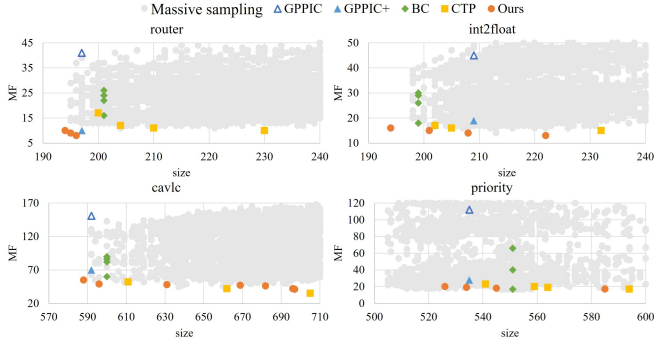


Fig. 6. The designs obtained by various compilers for small benchmarks.

Section IV-A2 on the whole netlist and reschedules the resulting netlist. For a fair comparison, we constrain the runtime of CTP to be slightly longer than that of our compiler for each benchmark.

Next, in Section V-A, we study the performance of various compilers in obtaining the Pareto-optimal designs. In Section V-B, we compare the EDPs of the ISs generated by various compilers. In Section V-C, we further study the performance of our MF-oriented resubstitution. In Section V-D, we perform a case study on *multiplier* benchmark to show the ability of our compiler in handling different array sizes.

#### A. Performance on Obtaining Pareto-Optimal Designs

Fig. 6 shows the designs obtained by various LiM compilers on MF-size plane for four small benchmarks, *router*, *int2float*, *cavlc*, and *priority*. For these benchmarks, we apply a massive sampling method to obtain the designs indicated by gray circles. Specifically, we randomly generate 5000 netlists for each benchmark and then use the schedulers in BCs [7]–[10] to schedule each of the netlist. The massive sampling method is very time-consuming. It takes 2 to 4 days to compile each benchmark, so we only test it on these small benchmarks.

For each benchmark, the hollow and solid triangles are the designs obtained by GPPIC and GPPIC+, respectively. They share the same netlist optimized by GPPIC [5], but scheduled with different schedulers, *i.e.*, the original one in [5] and the one proposed in [10], so they have the same size and the design scheduled by [10] has less MF. The four green diamonds are the designs obtained by the four BCs. They share the same

netlist optimized by iteratively applying *resyn2* commands, so they also have the same size. However, they are scheduled with different schedulers proposed in [7]–[10]. Instead of only obtaining one design, our compiler and its counterpart CTP can find multiple designs shown as orange circles and yellow rectangles, respectively.

As mentioned in Section III, it is important to find Pareto-optimal designs in terms of size and MF. Hence, a good compiler is the one that can find designs lying in the bottom left corner. From the figure, we can see that *in general, our compiler outperforms the existing works* [5], [7]–[10] in finding Pareto-optimal designs. It can even obtain better designs than the massive sampling method for all four benchmarks except *priority*. This minor disadvantage is acceptable since the massive sampling method takes a much longer runtime.

#### B. EDP When Using Multiple Arrays

In this section, we compare the EDPs of the ISs generated by various compilers when multiple arrays are used. We set a different array size for each benchmark as shown in the second column in Table II, so that we must use multiple arrays to compute the function. Given a design, we use the instruction generation method in [10] to generate the IS, inserting cross-array data copying instructions when needed. We use the method in [5] to estimate the EDP of the ISs by accumulating the energy and delay of each instruction and show the result in Table II. For simplicity, for each benchmark, we present the best BC IS, which is the IS with the least EDP generated from the designs obtained by the four BCs. As mentioned in Section III, for our compiler and CTP, we estimate the EDPs of the ISs generated from the Pareto-optimal designs obtained and pick the one with the least EDP.

The compiler giving the smallest EDP for each benchmark is marked in bold. We can see that our compiler outperforms the others for all benchmarks except two. Our compiler reduces the geometric mean of EDPs by 47.2%, 30.2%, 15.6%, and 16.0%, respectively, over GPPIC [5], GPPIC+, the best result of the compilers based on the basic flow, *i.e.*, [7]–[10], and the counterpart of our compiler.

As shown in Table II, GPPIC [5] and its improved version cannot achieve good performance, since their optimization command sequence only contains 10 commands. This shows

the importance of running the iterative improvement flow in the light blue box of Fig. 3 for  $R$  rounds in our compiler. For the BCs, the netlist size is effectively reduced: As we can see from the table, the geometric mean of the sizes of the netlists obtained by the BCs is the smallest among all methods. However, a design with a smaller size does not necessarily have less MF and lead to an IS with a smaller EDP. For example, for *sin* benchmark, our design has a larger size than the best BC design, but it generates an IS with 23% less EDP since it has less MF. This result further confirms the advantage of an iterative improvement flow, where a scheduled netlist is re-synthesized to improve its performance, over those one-pass flows. Our compiler also outperforms its counterpart which optimizes and schedules the whole netlist. This shows the strength of tight coupling of synthesis and scheduling in the iterative flow through our proposed techniques such as critical sub-netlist extraction and MF-oriented resubstitution, which accelerates each iteration and leads to more iterations under a runtime budget.

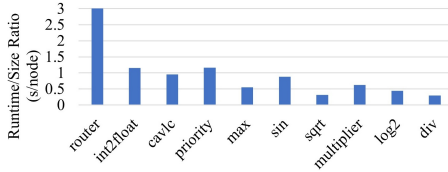


Fig. 7. Runtime/size ratio of the benchmarks with increasing sizes from left to right.

For runtime, our compiler can finish within 5 hours for all benchmarks, while GPPIC requires more than 8 hours to compile large benchmarks on a computer with a similar configuration. Our compiler is scalable since its key components, i.e., the existing logic optimization commands used in sub-netlist optimization, the scheduler [10], and our proposed logic optimization technique, MFResub, are all scalable. In Fig. 7, we plot the ratio of the runtime in seconds of our compiler over the size of each benchmark (#Nodes in Table I). We can see that the ratio does not grow with the benchmark size, confirming the scalability of our compiler.

### C. Performance of MFResub

In this section, we analyze the performance of our MF-oriented resubstitution MFResub. We generate 30 scheduled netlists for each benchmark by random optimization commands in Mockturtle [19] and the scheduler in [10], and apply MFResub to them. We categorize the outcome into the following six categories and show it as pie charts in Fig. 8. 1) No resub: no resubstitution is made in the process. 2) No change: resubstitution is made on the scheduled netlist but it

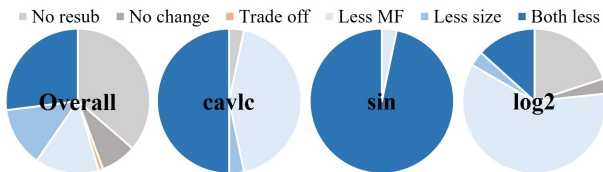


Fig. 8. The performance of MFResub.

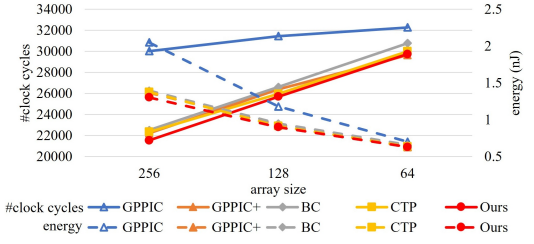


Fig. 9. Delay and energy of multiplier using different array sizes.

does not change its size or MF. 3) Trade off: in very rare cases, the resulting scheduled netlist has a smaller size but slightly larger MF. 4) Less MF: the resulting scheduled netlist has the same size and less MF. 5) Less size: the resulting scheduled netlist has the same MF and smaller size. 6) Both less: the resulting scheduled netlist has smaller size and less MF. Since our target is to find Pareto-optimal designs on size and MF, the latter three categories shown in blue colors all can be regarded as quality improvement. We can see from Fig. 8 that our MFResub can reduce both the size and MF for over a quarter of the  $30 \times 10$  scheduled netlists and improve the quality for more than half of them. Moreover, it can perform well on scheduled netlists of various sizes. For example, it improves the quality of most scheduled netlists of *cavlc*, *sin*, and *log2* benchmarks as shown in the figure.

### D. Case Study: Multiplier Using Different Array Sizes

In this section, we perform a case study using *multiplier* benchmark to show the ability of our compiler to handle different array sizes. As mentioned in Section III, with the Pareto-optimal designs obtained by our compiler, we can quickly estimate the EDP of the IS generated from each design when using different array sizes and pick the best one without the need to re-compile. We report the number of clock cycles and energy per bit when using different array sizes as shown in Fig. 9. As the array size decreases, the number of clock cycles required increases since more data copying instructions are needed, but the energy per bit decreases since the energy consumption of each instruction reduces. The figure shows that in general, our compiler can obtain better IS, i.e., the one with fewer number of clock cycles and less energy, than other methods.

## VI. CONCLUSION

In this work, we build a high-quality compiler for SIMD LiM. We reveal that scheduled netlists with smaller sizes do not necessarily have less MF. Hence, the target of our compiler is to find scheduled netlists on the Pareto frontier of size and MF. In order to achieve the target, we propose an iterative compiler with tight coupling of synthesis and scheduling, which is based on improving the critical sub-netlist identified by our algorithm using scheduling result and performing our proposed MF-oriented resubstitution that can reduce size and MF of a scheduled netlist without the need of re-scheduling. The experimental results show that our compiler can obtain scheduled netlists of better quality and the geometric mean of the EDPs of the ISs generated from them is reduced by 15.6% over the best prior method.



## REFERENCES

- [1] A. Pedram *et al.*, “Dark memory and accelerator-rich system optimization in the dark silicon era,” *IEEE Design & Test*, vol. 34, no. 2, pp. 39–50, 2017.
- [2] S. Kvatinsky *et al.*, “MAGIC–memristor-aided logic,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 61, no. 11, pp. 895–899, 2014.
- [3] N. Hajinazar *et al.*, “SIMDRAM: A framework for bit-serial SIMD processing using DRAM,” in *ASPLOS*, 2021, pp. 329–345.
- [4] P.-E. Gaillardon *et al.*, “The programmable logic-in-memory (PLiM) computer,” in *DATE*, 2016, p. 427–432.
- [5] C. Nie *et al.*, “XMG-GPPIC: Efficient and robust general-purpose processing-in-cache with XOR-Majority-Graph,” in *GLSVLSI*, 2023, pp. 183–187.
- [6] J. Wang *et al.*, “A 28-nm compute SRAM with bit-serial logic/arithmetic operations for programmable in-memory vector computing,” *IEEE J SOLID-STATE CIRCS*, vol. 55, no. 1, pp. 76–86, 2020.
- [7] M. Soeken *et al.*, “An MIG-based compiler for programmable logic-in-memory architectures,” in *DAC*, 2016, pp. 1–6.
- [8] R. Ben-Hur *et al.*, “SIMPLER MAGIC: Synthesis and mapping of in-memory logic executed in a single row to improve throughput,” *TCAD*, vol. 39, no. 10, pp. 2434–2447, 2020.
- [9] F. Wang *et al.*, “STAR: Synthesis of stateful logic in RRAM targeting high area utilization,” *TCAD*, vol. 40, no. 5, pp. 864–877, 2021.
- [10] X. Qian, C. Lv, Z. He, and W. Qian, “A recursive partition-based in-memory simd computation scheduler for memory footprint minimization,” *TCAD*, vol. 44, pp. 2105–2118, 2025.
- [11] M. R. H. Rashed *et al.*, “Logic synthesis for digital in-memory computing,” in *ICCAD*, 2022, pp. 1–9.
- [12] S. Shirinzadeh *et al.*, “Logic synthesis for in-memory computing using resistive memories,” in *ISVLSI*, 2018, pp. 375–380.
- [13] B. Brayton *et al.*, “ABC: An academic industrial-strength verification tool,” in *Proceedings of the 22nd International Conference on Computer Aided Verification*, 2010, p. 24–40.
- [14] S.-Y. Lee *et al.*, “A simulation-guided paradigm for logic synthesis and verification,” *TCAD*, vol. 41, no. 8, pp. 2573–2586, 2022.
- [15] X. Dong *et al.*, “NVSim: A circuit-level performance, energy, and area model for emerging nonvolatile memory,” *TCAD*, vol. 31, no. 7, pp. 994–1007, 2012.
- [16] N. Talati *et al.*, “Practical challenges in delivering the promises of real processing-in-memory machines,” in *DATE*, 2018, pp. 1628–1633.
- [17] A. Mishchenko *et al.*, “Scalable logic synthesis using a simple circuit structure,” in *IWLS*, 2006.
- [18] S. Yang, “Logic synthesis and optimization benchmarks,” Massachusetts Univ., Tech. Rep., 1988.
- [19] M. Soeken *et al.*, “The EPFL logic synthesis libraries,” 2022, arXiv:1805.05121v3.
- [20] H. Riener *et al.*, “On-the-fly and DAG-aware: Rewriting boolean networks with exact synthesis,” in *DATE*, 2019, pp. 1649–1654.
- [21] —, “Boolean rewriting strikes back: Reconvergence-driven windowing meets resynthesis,” in *ASP-DAC*, 2022, pp. 395–402.
- [22] L. Amarù *et al.*, “The EPFL combinational benchmark suite,” in *IWLS*, 2015.